Scikit-learn and Tour of Classifiers

First ML pipeline with Scikit-learn



Tour of Classifiers in scikit-learn

- Discuss a **selection** of **popular** and **powerful** machine learning algorithms **commonly** used in academia and industry
- Learn about the **differences** between several **supervised learning** algorithms for **classification** and their individual **strengths** and **weaknesses**
  - Perceptron
  - Logistic regression
  - Support vector machines
  - Decision trees
  - Random forest
  - K-nearest neighbours
- **First steps** with the scikit-learn library



Tour of Classifiers in scikit-learn

- Choosing an **appropriate** classification algorithm for a particular problem task requires practice
- **No Free Lunch theorem**: No single classifier works best across all possible scenarios
- “The **average performance** of **any pair** of algorithms across **all possible** problems is **identical**.” (Wolpert, D.H., Macready, W.G. (1997), No Free Lunch Theorems for Optimization, IEEE)
- If a specific algorithm seems to **outperform** another in a certain situation, it is a consequence of its **fit to the particular problem**, **not** the general superiority of the algorithm



Tour of Classifiers in scikit-learn

- Compare the performance of at least a **handful of different learning algorithms** to select the **best** model for the **particular** problem
- The problem at hand may be influenced by a number of contextual settings
  - Number of features
  - Number of samples
  - Distribution of the data
  - Amount of noise in the data
  - Whether classes are linearly separable or not
  - etc.



Tour of Classifiers in scikit-learn

- Performance of the classifier:
  - Computational performance
  - Predictive power

Summary of the five main steps in training an ML algorithm

1. **Selecting features** and **collecting training samples**
2. Choosing a **performance metric**
3. Choosing a **classifier** and an **optimisation algorithm**
4. Evaluating the **performance of the model**
5. Tuning the **algorithm**

Training a perceptron with scikit-learn

Training a Perceptron on the Iris data set

- Imports

```
# ============================================================================
# Import modules
# ============================================================================

import matplotlib.pyplot as plt
import numpy as np
from sklearn import datasets
from sklearn.inspection import DecisionBoundaryDisplay
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

scikit_learn_intro.ipynb

Training a Perceptron on the Iris data set

- Use two features (easy visualization)

```
# ============================================================================
# Load data and select features
# ============================================================================

iris = datasets.load_iris()
X = iris.data[:, [2, 3]]
y = iris.target
```

scikit_learn_intro.ipynb



Training a Perceptron – class label encoding

- Class labels: Scikit-learn also works with strings as class labels
- Converting to integers is recommended due to memory and runtime performance

Multi-class classification with OvR



Multi-class

We only discussed binary classifiers, what about multiple classes?

OvR (One-vs-Rest) also called OvA (One-vs-All)

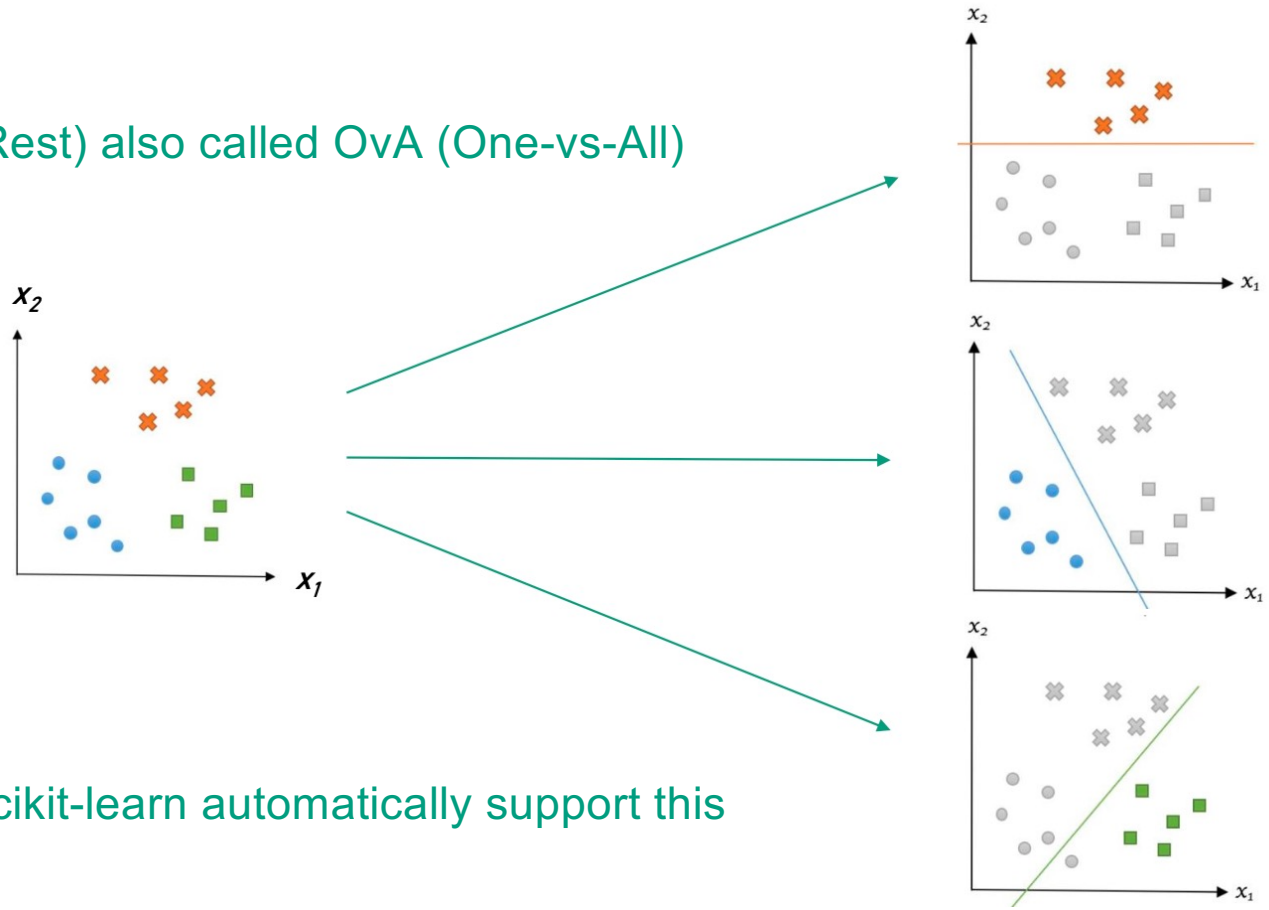
- A technique that allows us to extend a binary classifier to multi-class problems
- Using OvA, we can **train one classifier per class**, where a particular class is treated as the positive class and the samples from all other classes are considered as members of the negative class

Classification of a new sample:

- Use the *n* trained classifiers, where *n* is the number of class labels
- Assign the class label with the highest confidence
- We usually want a classifier for this that can output probabilities (e.g. logistic regression)
- Perceptron: Choose the class label that is associated with the largest net input value *z*

Multi-class

OvR (One-vs-Rest) also called OvA (One-vs-All)



Classifiers in scikit-learn automatically support this

Test/Train split

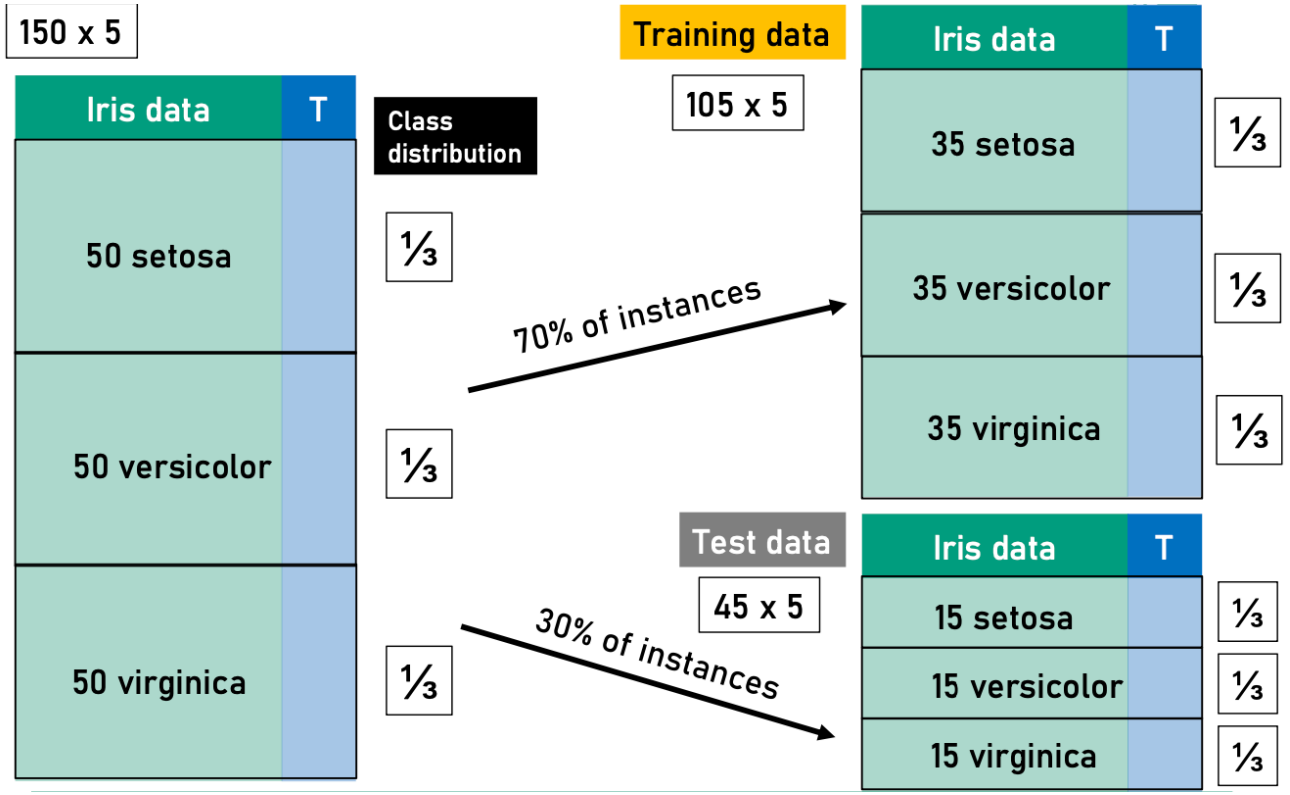
Training a Perceptron – test/train split

- Split the data into separate training and test datasets
- More details later in “Best practices for evaluation and hyperparameter tuning”
- Use the `train_test_split` function from scikit-learn's `model_selection` module
- Randomly split the *X* and *y* arrays into 30 percent test data (45 samples) and 70 percent training data (105 samples)

```
# Split data into training and test data (70% training, 30% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1, stratify=y
)
```

`scikit_learn_intro.ipynb`

Training a Perceptron – test/train split



Training a Perceptron – test/train split

- Properties of the `train_test_split` function:
  - **Shuffles the dataset** internally **before** splitting → we don't need to worry about the ordering of the samples prior to splitting
  - Provides a `random_state` parameter for a **fixed random seed** (default: `random_state=None`) → «**reproducible**» shuffling of samples (handy when training multiple times)
  - Built-in support for **stratification** with `stratify=y → train_test_split` returns training and test subsets that have the **same proportions of class labels** as the **input dataset**

```
# Split data into training and test data (70% training, 30% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1, stratify=y
)
```

Training a Perceptron – test/train split

- Checking the distribution of the classes in the training and test set with numpy:

```
# Show distribution of classes in input data, training data and test data
print(f"Labels counts in y: {np.bincount(y)}")
print(f"Labels counts in y_train: {np.bincount(y_train)}")
print(f"Labels counts in y_test: {np.bincount(y_test)}")
```

- Output:

```
Labels counts in y: [50 50 50]
Labels counts in y_train: [35 35 35]
Labels counts in y_test: [15 15 15]
```

scikit_learn_intro.ipynb

Training a Perceptron – feature scaling

- Many machine learning and optimisation algorithms also require feature scaling for optimal performance
- Standardise the features using the `StandardScaler` class from scikit-learn's `preprocessing` module

```
# Initialise standard scaler and compute mean and stddev from training data
sc = StandardScaler()
sc.fit(X_train)

# Transform (standardise) both X_train and X_test with mean and stddev from
# training data
X_train_sc = sc.transform(X_train)
X_test_sc = sc.transform(X_test)
```

`scikit_learn_intro.ipynb`

Training a Perceptron – feature scaling

Person	Height (cm)	Weight (kg)	Shoe size
Person A	174	55	46
Person B	188	92	45
Person C	158	65	42
Person D	202	110	49
Person E	171	96	44
Person F	193	79	48
Mean	181	82.833333	45.6667
STD	16.198765	20.507722	2.58199

Original data

Person	Height (cm)	Weight (kg)	Shoe size
Person A	-7	-27.833333	0.33333
Person B	7	9.1666667	-0.66667
Person C	-23	-17.833333	-3.66667
Person D	21	27.166667	3.33333
Person E	-10	13.166667	-1.66667
Person F	12	-3.8333333	2.33333
Mean	0.00	0.00	0.00
STD	16.198765	20.507722	2.58199

Centred data

Person	Height (cm)	Weight (kg)	Shoe size
Person A	-0.4321317	-1.3572123	0.1291
Person B	0.4321317	0.4469861	-0.2582
Person C	-1.4198613	-0.8695911	-1.42009
Person D	1.2963951	1.3247043	1.29099
Person E	-0.617331	0.6420346	-0.6455
Person F	0.7407972	-0.1869215	0.9037
Mean	0.00	0.00	0.00
STD	1	1	1

Standardised (scaled) data

`scikit_learn_intro.ipynb`

Training a Perceptron – feature scaling

- Important: Scale the test and training data with the scaling obtained from the training data!
- Using the fit method, StandardScaler estimates the parameters μ (sample mean) and σ (standard deviation) for **each feature** dimension from the **training data**
- Calling the transform method: Standardises the training and test data using the estimated parameters μ and σ acquired from the **training data** → the values from the training and test data are comparable to each other

```
# Initialise standard scaler and compute mean and stddev from training data
sc = StandardScaler()
sc.fit(X_train)

# Transform (standardise) both X_train and X_test with mean and stddev from
# training data
X_train_sc = sc.transform(X_train)
X_test_sc = sc.transform(X_test)
```

scikit_learn_intro.ipynb

Training the model

Training a Perceptron

- Load the `Perceptron` class from the `linear_model` module
- Initialise a new Perceptron object
  - The parameter `eta0` defines the learning rate
  - The parameter `max_iter` defines the number of epochs (passes over the training set)
- Train the model using the `fit` method

```
# Initialise the model
ppn = Perceptron(max_iter=40, eta0=0.1, random_state=1)
ppn.fit(X_train_sc, y_train)
```

`scikit_learn_intro.ipynb`

Training a Perceptron

- Finding an appropriate learning rate requires some experimentation
- If the learning rate is too large, the algorithm will overshoot the global cost minimum
- If the learning rate is too small, the algorithm requires more epochs until convergence → can make the learning slow, especially for large datasets
- The `random_state` parameter ensures the reproducibility of the initial shuffling of the training dataset after each epoch

```
# Initialise the model
ppn = Perceptron(max_iter=40, eta0=0.1, random_state=1)
ppn.fit(X_train_sc, y_train)
```

`scikit_learn_intro.ipynb`

Evaluation/Prediction
Performance metrics: Accuracy

Training a Perceptron – Predictions

```
# Predict classes for samples in test set and print number of misclassfications
y_pred = ppn.predict(X_test_sc)
print("Misclassified samples: {0}".format((y_test != y_pred).sum()))
```

```
Misclassified samples: 1
```

```
# Print accuracy computed from predictions on the test set
print("Accuracy: {0:.2f}".format(accuracy_score(y_test, y_pred)))


# Print accuracy computed from predictions on the test set
print("Accuracy: {0:.2f}".format(ppn.score(X_test_sc, y_test)))
```

Misclassification error: 1 out of 45→1 / 45 ≈ 0.022 or approx. 2.2%
Often the **accuracy** is reported instead of the misclassification error:
1 – error = 0.978 = 97.8%

`scikit_learn_intro.ipynb`

Training a Perceptron – Accuracy

Simple performance metric; the method `score` also outputs accuracy

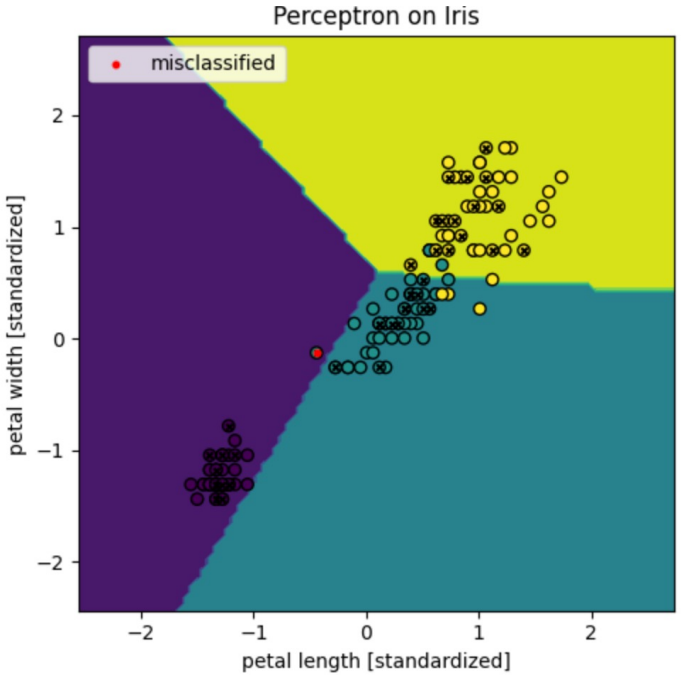
```
# Print accuracy computed from predictions on the test set
print("Accuracy: {0:.2f}".format(accuracy_score(y_test, y_pred)))


# Print accuracy computed from predictions on the test set
print("Accuracy: {0:.2f}".format(ppn.score(X_test_sc, y_test)))
```

`scikit_learn_intro.ipynb`

Training a Perceptron – Plot the decision boundary

- The three flower classes cannot be perfectly separated by a linear decision boundary
- The algorithm would go on forever, scikit-learn has a stopping criterion if no improvement



`scikit_learn_intro.ipynb`

Recap

Agenda



- Repetition
  - Gradient descent vs. Newtons method
  - One-vs-Rest/One-vs-All
  - Test/train split
  - Feature scaling with StandardScaler
  - Training a model and using it for predictions
- Exercises
- Example: Model selection
- Quizzes

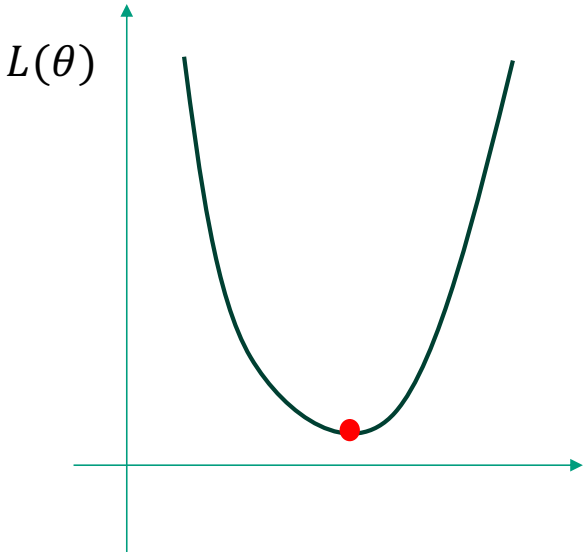
Gradient descent

Finds the minimum of a function

Newton's method

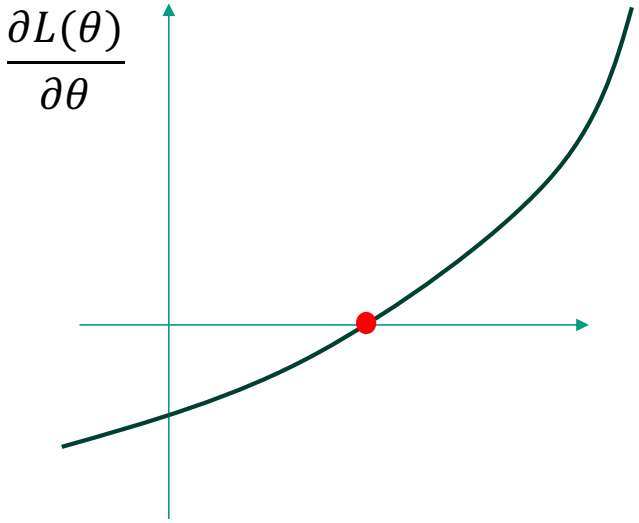
Finds the root (zero-point) of a function

Two different algorithms, different goals!



$$\boldsymbol{\theta}^{n+1} = \boldsymbol{\theta} - \eta \frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$$

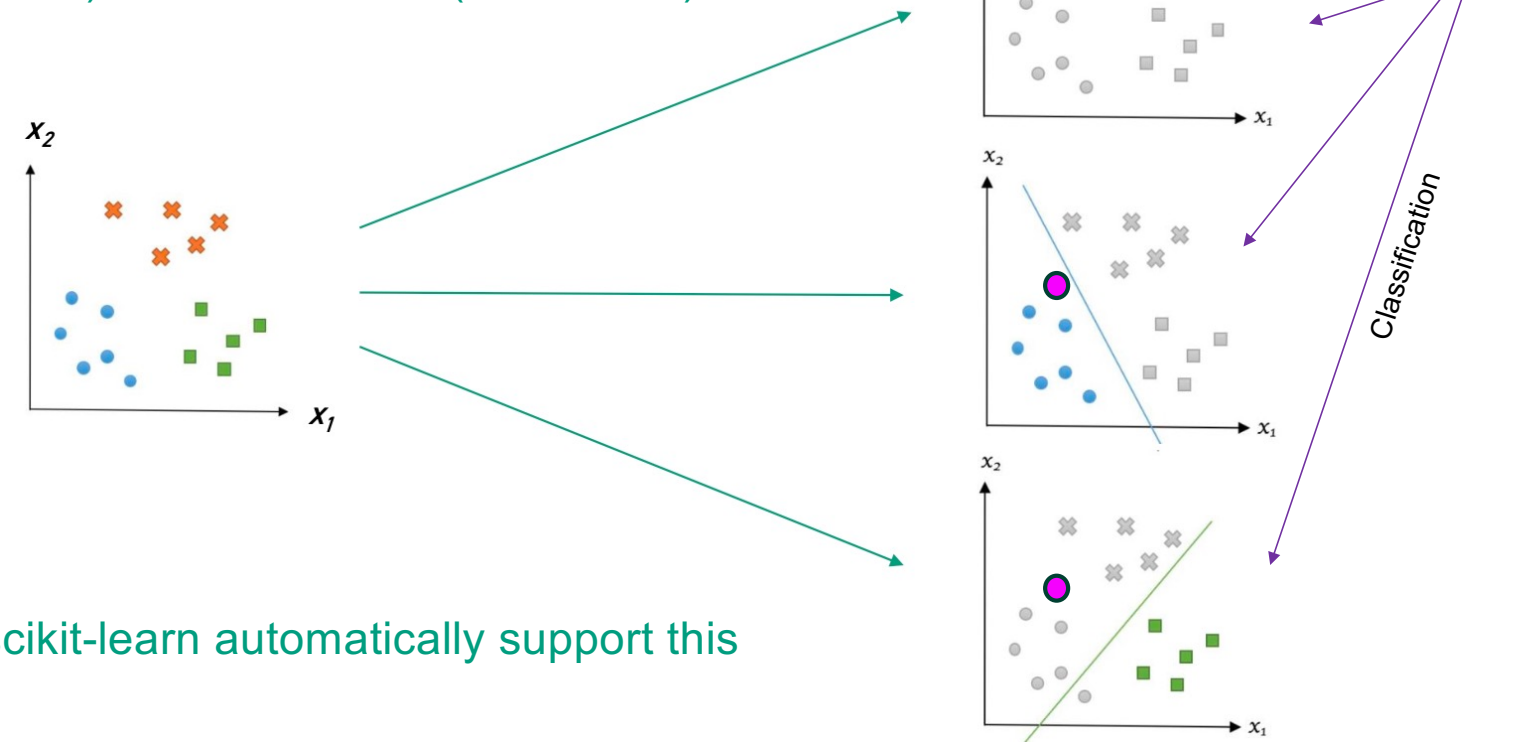
Connection: The **function derivative** is **zero** at the **minimum**!



$$\theta^{n+1} = \theta^n - \frac{f(\theta^n)}{f'(\theta^n)} = \boldsymbol{\theta}^n - \left(\frac{\partial^2 L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}}\right)^{-1} \frac{\partial L(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} (\boldsymbol{\theta}^n)$$

Multi-class

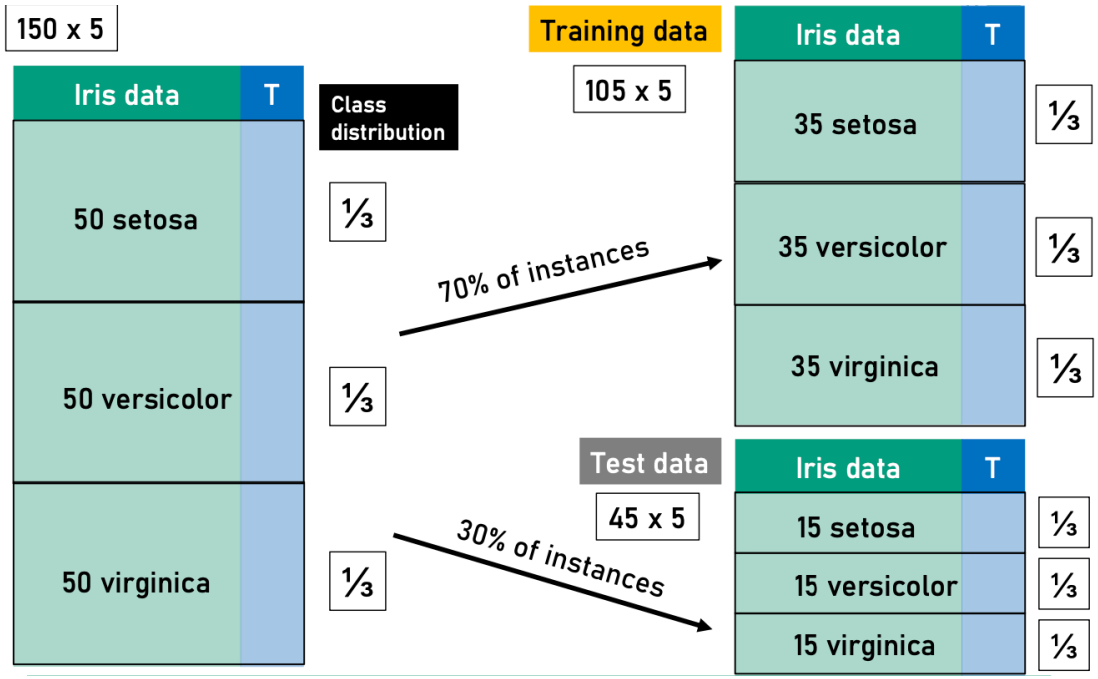
OvR (One-vs-Rest) also called OvA (One-vs-All)



Classifiers in scikit-learn automatically support this

Test/train split

Randomly split the *X* and *y* arrays into 30 percent test data and 70 percent training data




```
# Split data into training and test data (70% training, 30% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1, stratify=y
)
```

scikit_learn_intro.ipynb

Feature scaling

- Important: Scale the test and training data with the scaling obtained from the training data!
- StandardScaler.fit: Estimates the parameters μ (sample mean) and σ (standard deviation).
- StandardScaler.transform: Standardises the training and test data using the estimated parameters μ and σ acquired from the **training data**.

```
# Initialise standard scaler and compute mean and stddev from training data
sc = StandardScaler()
sc.fit(X_train)

# Transform (standardise) both X_train and X_test with mean and stddev from
# training data
X_train_sc = sc.transform(X_train)
X_test_sc = sc.transform(X_test)
```

scikit_learn_intro.ipynb

Training a model and using it for predictions

- Import the model class from e.g. Scikit-learn
- Initialise the model (specify values of hyperparameters like eta0 and max_iter)
- Train the model using the `fit` method on the training set
- Make predictions on the test set using the `predict` method

`scikit_learn_intro.ipynb`

```
# Initialise the model
ppn = Perceptron(max_iter=40, eta0=0.1, random_state=1)
ppn.fit(X_train_sc, y_train)
```

```
# Predict classes for samples in test set and print number of misclassfications
y_pred = ppn.predict(X_test_sc)
print("Misclassified samples: {0}".format((y_test != y_pred).sum()))
```

```
# Print accuracy computed from predictions on the test set
print("Accuracy: {0:.2f}".format(accuracy_score(y_test, y_pred)))
```



Training a Perceptron – Exercise

- Apply what we saw to the Wine data set
- Train a `Perceptron` on the Wine data (`datasets.load_wine`) that is integrated in scikit-learn
  - Use two features of the dataset: '`alcohol`' and '`hue`'
  - Train – test – split: 60 percent of the data for training, random state should be set to 5
  - Perceptron: Set the maximum number of iterations to 100
  - Perceptron: Set the learning rate to 0.05
  - Perceptron: Set the random state to 77
  - Plot the decision regions for the training and test data

  - **Extra** (look in the online documentation of the Perceptron): **1.** What is the test accuracy? **2.** How many epochs for training? **3.** How often were the weights updated? **4.** What are the final weights?

Training a Logistic regression – Exercise 2

- Train a `LogisticRegression` on the Wine data set (`datasets.load_wine`)
  - Use two features of the dataset: '`alcohol`' and '`hue`'
  - Train – test – split: 60 percent of the data for training, random state should be set to 5
  - `LogisticRegression`: Set the maximum number of iterations to 100
  - `LogisticRegression`: Set the random state to 77
  - Plot the decision regions for the training and test data

  - **Extra** (look in the online documentation of Logistic regression): **1.** What is the test accuracy? **2.** How many epochs for training? **3.** What are the final weights?

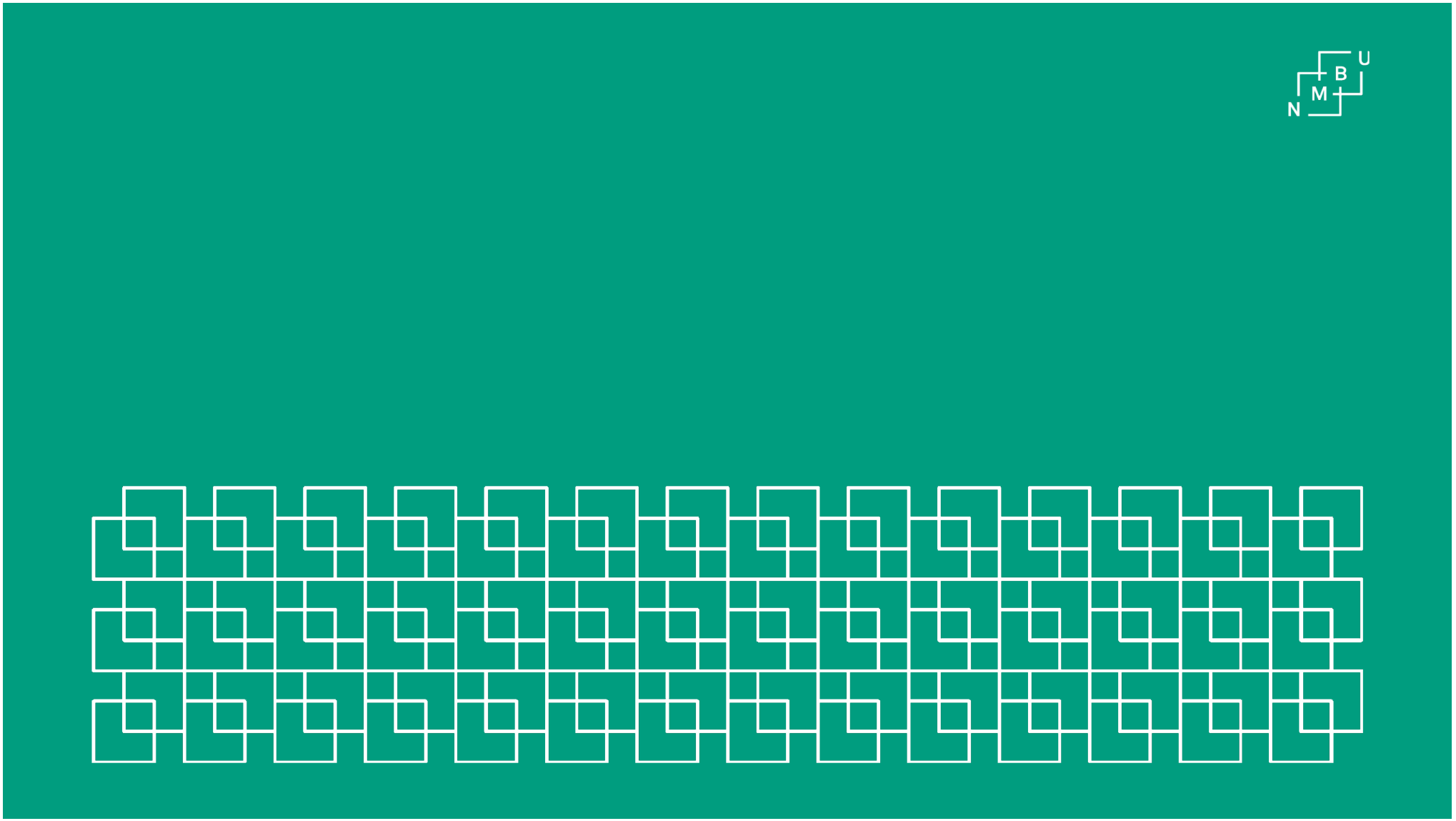


Example: Model selection

- Dataset: fetal_health.csv
- Models:
  - Perceptron
  - Logistic regression
  - Adaline
- Criteria for comparison:
  - Test set classification accuracy
  - Computational time

`scikit_learn_fetal_health_model_selection.ipynb`

Quizzes


N
M
B
U